



UNIVERSITÀ DI PISA

Botnet Detection

Artificial Intelligence for Cybersecurity

A.Y 2025/26

Andrea Conte
Matricola 675683

16/09/2026

Goal

Comparison of three models for botnet classification and detection.



Random Forest



Decision Tree



Logistic Regression

Data Acquisition & Data Exploration

A dataset containing network traffic traces, obtained from the kaggle.com platform, was used for model training and evaluation.

The dataset includes both legitimate traffic and malicious activity.

Each instance within the dataset is strictly labeled to distinguish benign traffic, classified as:

- **Normal/Background**
- **Botnet**

File: capture20110815-2.csv

Data Exploration

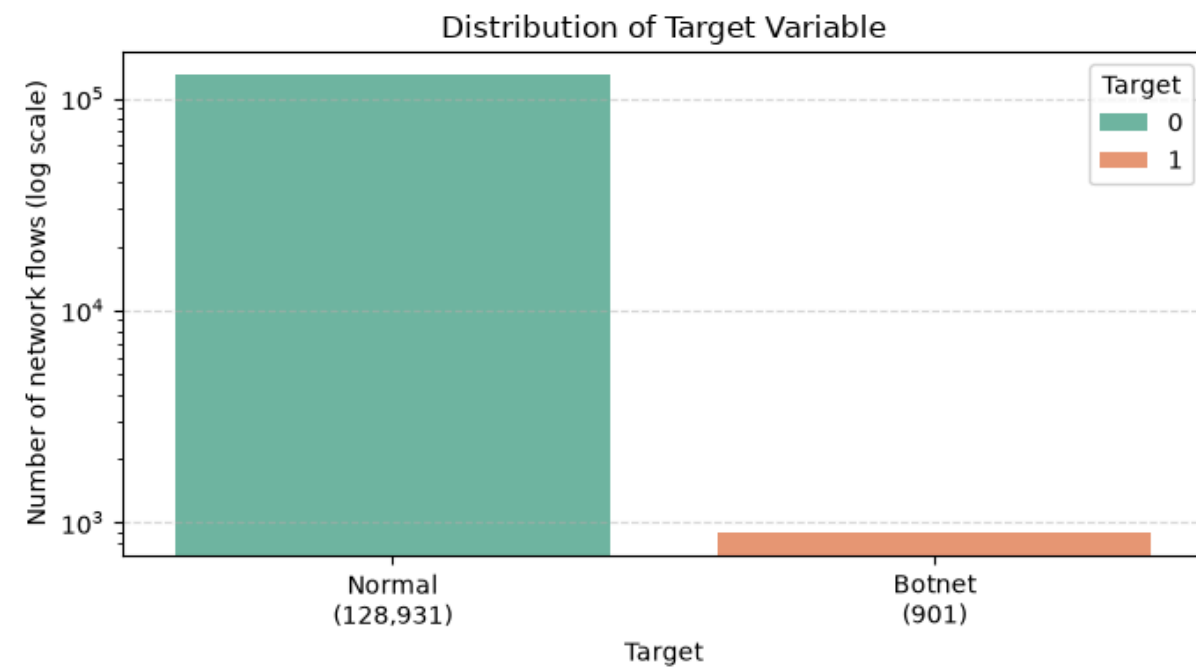
```
<class 'pandas.DataFrame'>
RangeIndex: 129832 entries, 0 to 129831
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   StartTime  129832 non-null  str
1   Dur         129832 non-null  float64
2   Proto       129832 non-null  str
3   SrcAddr     129832 non-null  str
4   Sport       129035 non-null  str
5   Dir         129832 non-null  str
6   DstAddr     129832 non-null  str
7   Dport       129455 non-null  str
8   State       129832 non-null  str
9   sTos        128974 non-null  float64
10  dTos        122936 non-null  float64
11  TotPkts     129832 non-null  int64
12  TotBytes    129832 non-null  int64
13  SrcBytes    129832 non-null  int64
14  Label       129832 non-null  str
dtypes: float64(3), int64(3), str(9)
memory usage: 14.9 MB
```

The dataset comprises a total of **129,832 instances** and **15 features**.

Data integrity was verified by counting non-null values for each attribute while simultaneously identifying the respective data types.

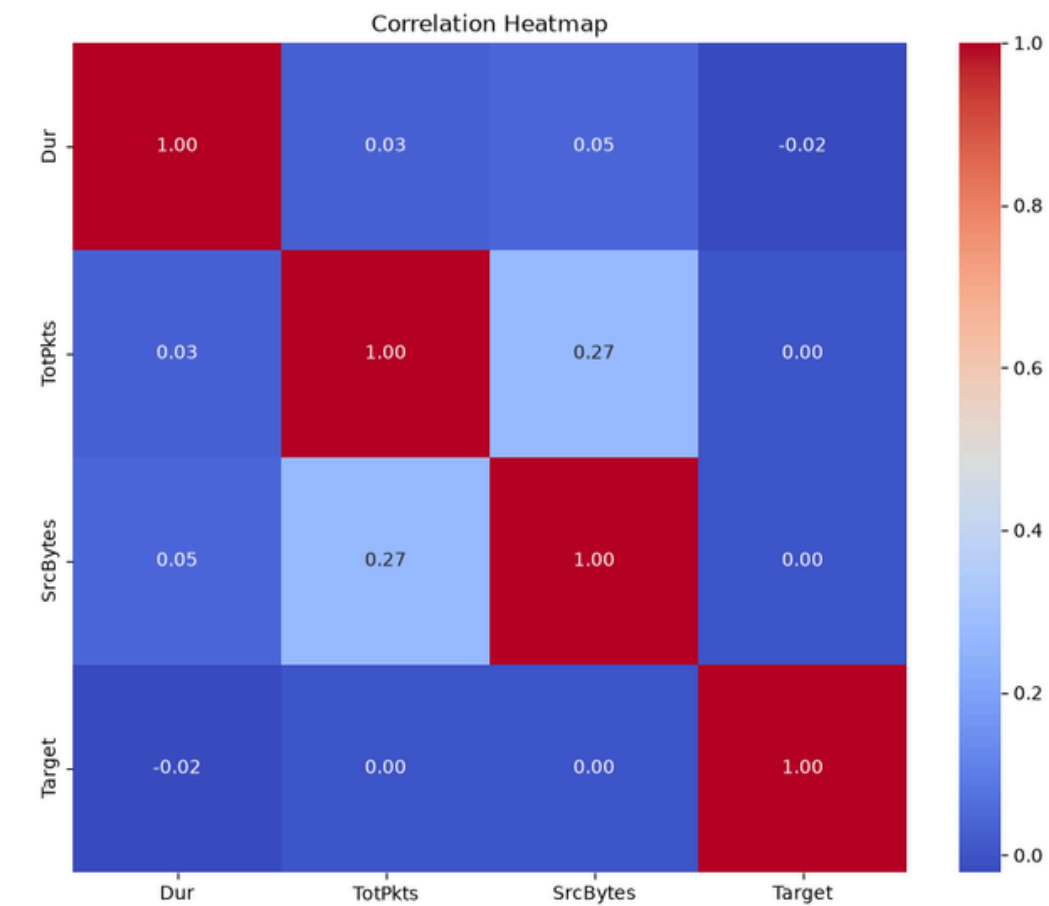
Data Exploration

In this phase, two fundamental structural characteristics of the dataset are highlighted.



Extreme disproportion between the benign and malicious classes:

- **128,931** instances of Normal/Background
- **901** Botnet instances



The correlation matrix reveals the absence of strong direct linear dependencies between the individual numerical features and the Target variable

Data Preprocessing

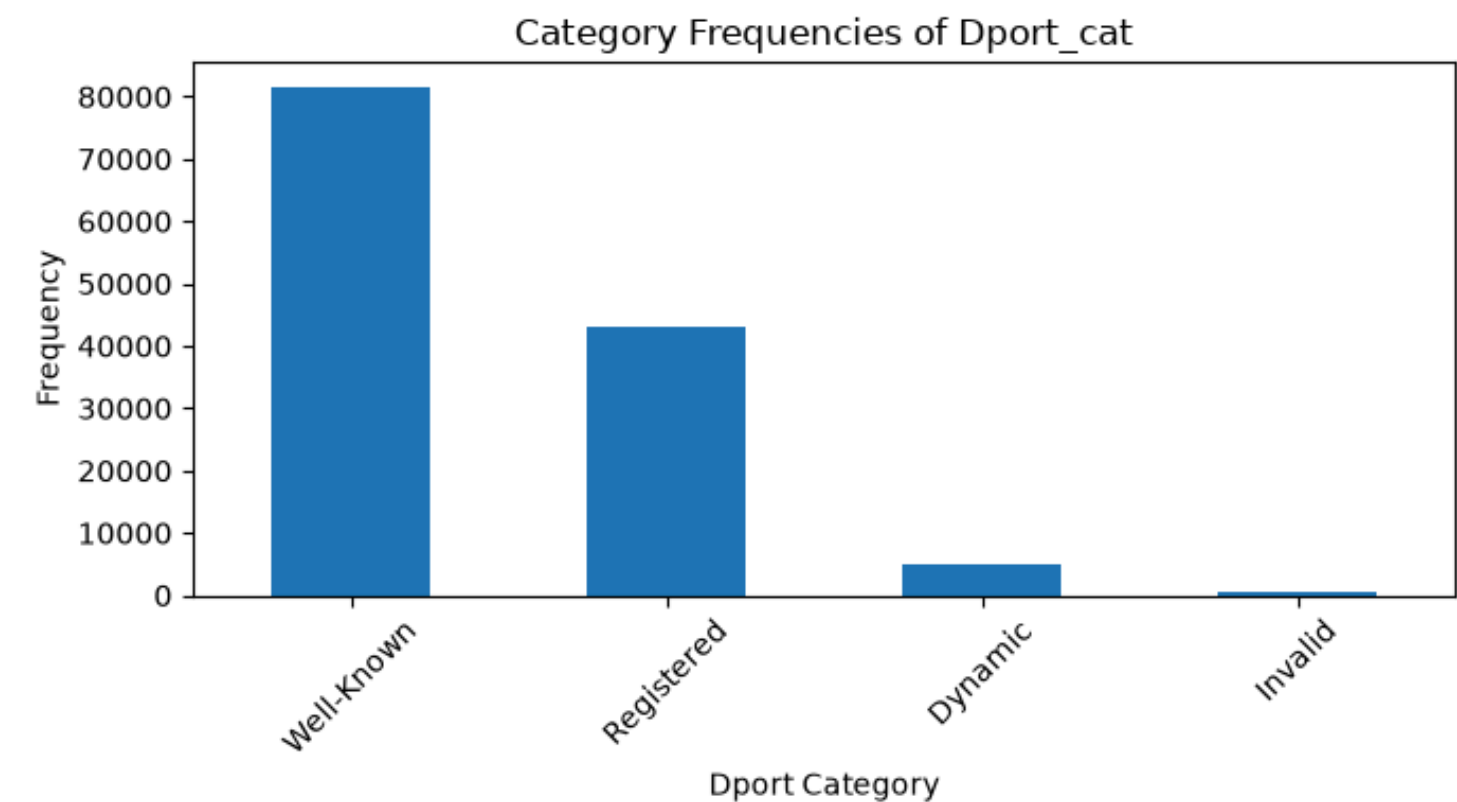
```
...  
--- Categorical features check ---  
Feature 'Proto': 11 unique values  
Feature 'State': 155 unique values  
Feature 'Dport': 12013 unique values  
Feature 'Dport_cat': 4 unique values
```

The Dport (destination port) feature has a high cardinality with **12,013** unique values.

To optimize the modeling, the Dport_cat feature was created, grouping the ports according to IANA (Well-Known, Registered, Dynamic) standards.

However, I decided to conduct an experiment in parallel.

Will the categorization of Dports be useful for classification purposes?



I converted Dport to categorical variables because network ports do not represent a scalar quantity but a label.

Mathematical operations cannot be carried out on them
For example port 53 (DNS) is not greater than 123 (NTP).

Data Preprocessing

The Chi-Square test was performed with respect to the Target variable.

Protocol (Proto) and State (State): both characteristics show a highly significant correlation with the presence of malicious traffic ($p < 0.001$), confirming that specific connection behaviors characterize botnet activity.

Dport and Dport_cat: the transformation from raw Dport to Dport_cat maintains a statistically determinant association with the destination, confirming that semantic grouping does not result in any loss of essential information power, but reduces computational complexity.

All 5 categorical variables have a total dependence on the presence of Botnet traffic.

```
... Chi2 Statistic: 2375.6462
    p-value: 0.0000e+00
    Result: Statistically significant dependence (p < 0.05)

    Chi2 Statistic: 13451.9699
    p-value: 0.0000e+00
    Result: Statistically significant dependence (p < 0.05)

    Chi2 Statistic: 50119.4916
    p-value: 0.0000e+00
    Result: Statistically significant dependence (p < 0.05)

    Chi2 Statistic: 709.7200
    p-value: 1.6381e-153
    Result: Statistically significant dependence (p < 0.05)
```

Data Preprocessing

...

```
--- Numerical features check ---  
Feature 'Dur': 59844 unique values  
Feature 'TotPkts': 802 unique values  
Feature 'SrcBytes': 7041 unique values
```

The check was also performed on the numerical features.

The other columns were discarded.

```
cols_to_drop = ['Label', 'Target', 'Dir', 'SrcAddr', 'DstAddr', 'sTos', 'dTos', 'Sport', 'StartTime', 'TotBytes']
```

Data Processing

For numeric variables I handled any missing values by imputing the median and applied the StandardScaler.

```
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
```

For categorical variables we impute an explicit category (missing) for the missing data and transform them with the OneHotEncoder.

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant', fill_value='missing')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=True))
])
```

The use of **sparse_output=True** was fundamental.
By storing the matrix in sparse format you can avoid memory crashes (out-memory).

Data Processing / Modelling

```
preprocessor_raw = ColumnTransformer(  
    transformers=[  
        ('num', numerical_transformer, numerical_features),  
        ('cat', categorical_transformer, categorical_features_raw)  
    ],  
    remainder='drop'  
)  
  
preprocessor_cat = ColumnTransformer(  
    transformers=[  
        ('num', numerical_transformer, numerical_features),  
        ('cat', categorical_transformer, categorical_features_cat)  
    ],  
    remainder='drop'  
)
```

Finally, I encapsulated everything in two separate instances of ColumnTransformer:

- one to analyze the raw features
- one to test the effectiveness of IANA grouping on ports.

Data Processing / Modelling

Configuration of Classifiers



Random Forest

```
class_weight='balanced'  
max_depth=None  
random_state=42
```



Decision Tree

```
class_weight='balanced'  
max_depth=None  
random_state=42
```



Logistic Regression

```
class_weight='balanced'  
max_iter=1000  
random_state=42
```

Model Evaluation

Evaluation Strategy (StratifiedKFold): 10-fold stratified cross-validation (n_splits=10, shuffle=True). Preserve the proportion of Botnet classes in each fold and prevent data leakage.

Calculation of Performance Metrics: for each model extracts predictions and probabilities via `cross_val_predict`, calculating the average and standard deviation of:

- Precision
- Recall
- F1-Score
- ROC-AUC on the Botnet class

Model Evaluation

... Evaluation in progress for Random Forest...
Evaluation in progress for Logistic Regression...
Evaluation in progress for Decision Tree...

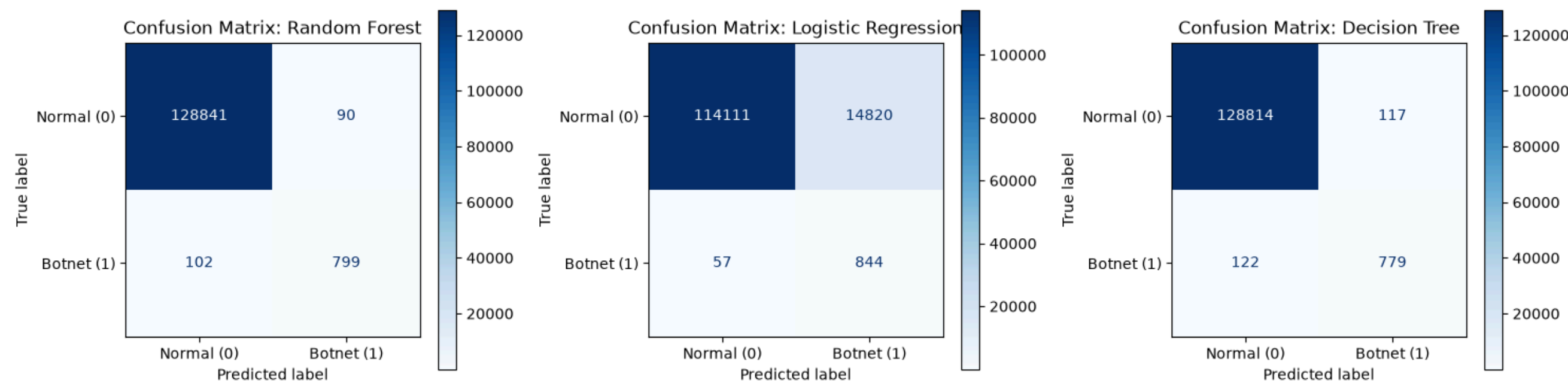
===== COMPARATIVE PERFORMANCE TABLE =====				
	Model	Precision (Botnet)	Recall (Botnet)	F1-Score (Botnet) ROC-AUC
	Random Forest	0.8993 ± 0.0192	0.8868 ± 0.0309	0.8926 ± 0.0156 0.9714 ± 0.0128
	Logistic Regression	0.0539 ± 0.0021	0.9368 ± 0.0186	0.1020 ± 0.0039 0.9565 ± 0.0123
	Decision Tree	0.8699 ± 0.0234	0.8646 ± 0.0423	0.8666 ± 0.0248 0.9318 ± 0.0232
=====				

... Evaluation in progress for Random Forest (Categorized)...
Evaluation in progress for Logistic Regression (Categorized)...
Evaluation in progress for Decision Tree (Categorized)...

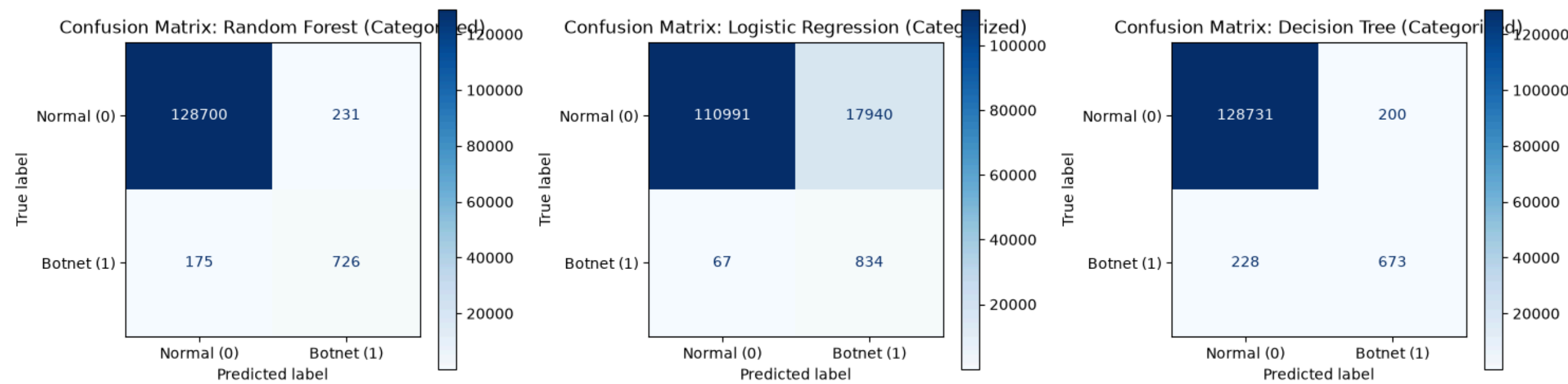
===== COMPARATIVE PERFORMANCE TABLE =====				
	Model	Precision (Botnet)	Recall (Botnet)	F1-Score (Botnet) ROC-AUC
	Random Forest (Categorized)	0.7605 ± 0.0468	0.8057 ± 0.0446	0.7816 ± 0.0375 0.9596 ± 0.0086
	Logistic Regression (Categorized)	0.0444 ± 0.0013	0.9256 ± 0.0217	0.0848 ± 0.0024 0.9382 ± 0.0151
	Decision Tree (Categorized)	0.7728 ± 0.0301	0.7469 ± 0.0461	0.7583 ± 0.0224 0.8700 ± 0.0245
=====				

Model Evaluation

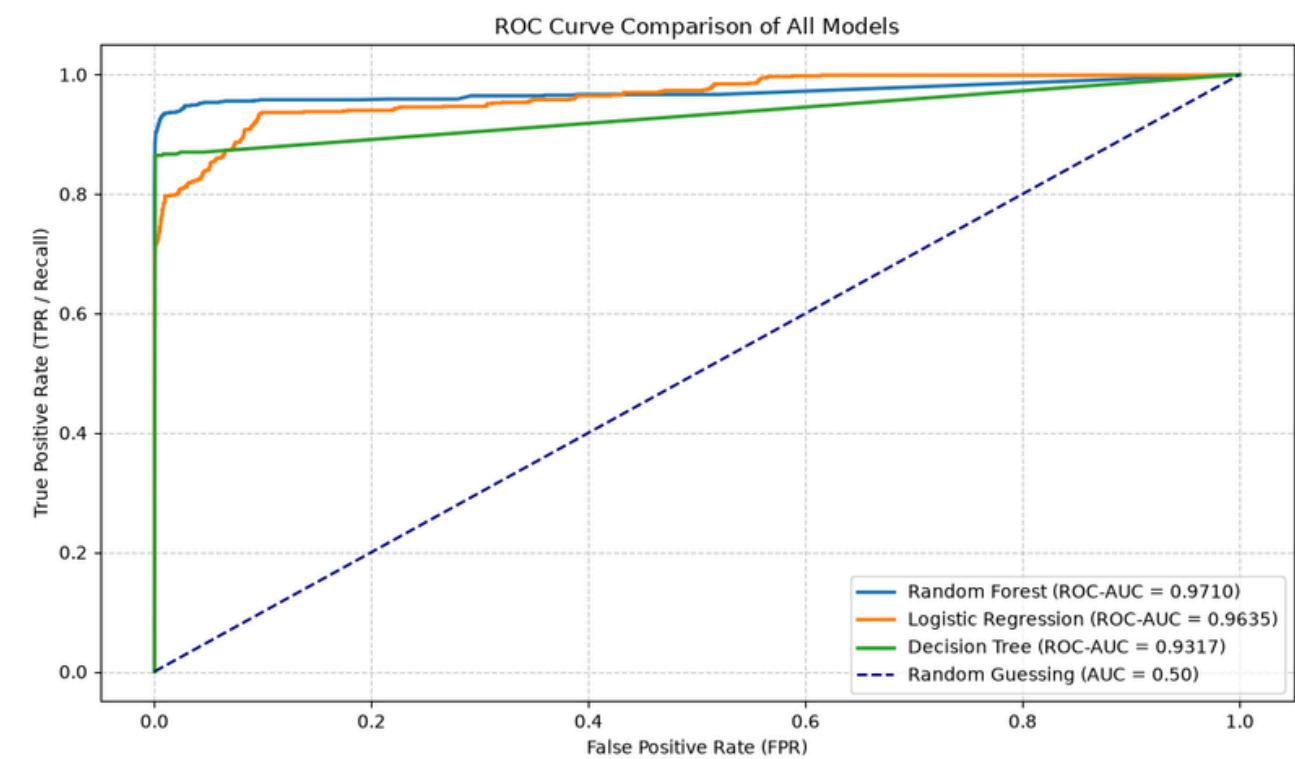
Experiment with Dport (raw)



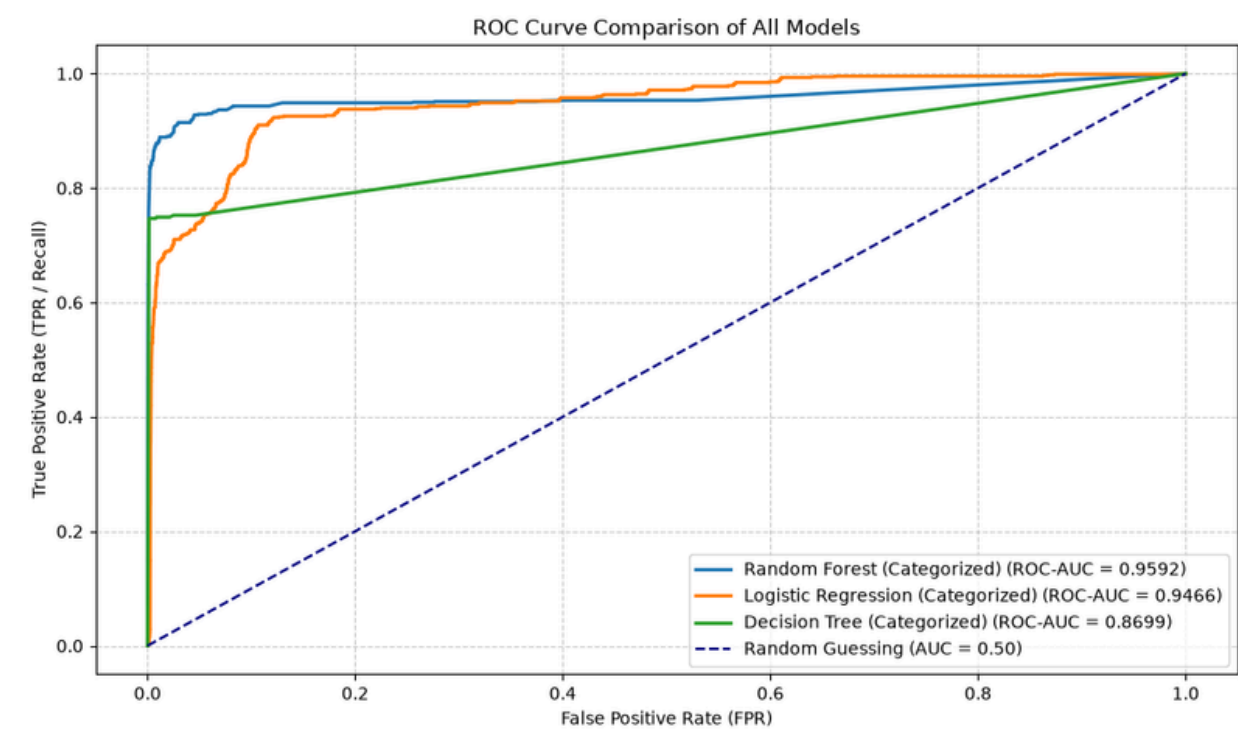
Experiment with Dport_cat



Model Evaluation



Experiment with Dport (raw)



Experiment with Dport_cat

Conclusion

In summary, Random Forest is the model chosen for this Botnet Detection task.

It guarantees the best balance between high botnet identification (Recall) and a reduced number of improper alerts (Precision/False Positives).

Next Step: test how resistant the system is to evasion techniques used by attackers to disguise botnet traffic as normal traffic.

Thank you all

References

Dataset: <https://github.com/shramos/Awesome-Cybersecurity-Datasets#network-traffic>

Full dataset: <https://www.stratosphereips.org/datasets-ctu13/>